

# National University of Computer and Emerging Sciences, Karachi

FAST School of Computing | Spring 2026

CS2006 - Operating Systems

## 1. Project Title & Group Information

|                      |                                                     |
|----------------------|-----------------------------------------------------|
| <b>Project Title</b> | Banker's Algorithm for Deadlock Avoidance Simulator |
| <b>Course</b>        | CS2006 - Operating Systems                          |
| <b>Semester</b>      | Spring 2026                                         |
| <b>Member 1</b>      | Murtaza Hunaid Terai   24K-0782                     |
| <b>Member 2</b>      | Adnan Hatim   24K-0656                              |

## 2. Proposed Project Description

This project implements a fully functional simulation of the Banker's Algorithm, a classical deadlock avoidance algorithm in operating systems. The simulator models a multi-process, multi-resource environment where the OS acts as a 'banker' granting or denying resource requests based on whether the resulting system state remains safe.

### How It Works

The system maintains three core data structures: the Allocation matrix (resources currently held by each process), the Maximum matrix (peak demand of each process), and the Need matrix (remaining resources each process may still request). When a process submits a resource request, the simulator first checks whether the request is within the declared need and whether sufficient resources are available. A hypothetical allocation is then performed and the Safety Algorithm is executed. If a safe sequence is found, the allocation is committed; otherwise, the process must wait and the state is rolled back.

### Functional Features

#### **Mandatory Features:**

- Representation of resource types: Support for multiple resource types with configurable total instances per type.
- Allocation, Maximum, and Need matrices: Dynamic construction and display of all three matrices as processes are added and resources are allocated.
- Safety algorithm implementation: Step-by-step execution of the Safety Algorithm to produce a valid safe sequence.
- Request handling logic: Processing of runtime resource requests per process, including validation against available resources and declared maximums.
- Deadlock avoidance verification: Proactive verification before each allocation to ensure the system never enters an unsafe state.

#### **Additional Features:**

- Graphical visualization: Real-time visual representation of the resource allocation state, including matrix tables and safe sequence output.

- Interactive input: Step-by-step interactive mode allowing users to configure processes, resources, and issue requests manually at runtime.
- Logging of safe/unsafe states: Every state transition is timestamped and written to a persistent log file for review and debugging.
- Rollback simulation: When a request leads to an unsafe state, the simulator automatically rolls back the hypothetical allocation and displays the rejected state with reasoning.
- Performance analysis: Metrics collection on requests granted, denied, and average time per safety check, displayed as a summary report at exit.

### 3. Plan of Work

The following outlines the short-term development plan for the project, divided between the two team members based on functional responsibilities.

#### Member 1: Murtaza Hunaid Terai

- Implement core data structure initialization: Allocation, Maximum, Need, and Available matrices with support for configurable processes and resource types.
- Implement the Safety Algorithm: Produce a safe execution sequence and verify overall system safety.
- Implement request handling logic: Validate and process resource requests, performing hypothetical allocation and invoking the safety check before committing.
- Implement rollback simulation: Detect and revert unsafe hypothetical allocations with logged reasoning.
- Implement performance analysis module: Collect and display metrics including request counts, denial rates, and per-check execution times.

#### Member 2: Adnan Hatim

- Implement resource type representation: Define and manage resource classes, their total counts, and available instances throughout the simulation lifecycle.
- Implement deadlock avoidance verification: Integrate pre-allocation safety checks before any resource grant is finalized.
- Implement graphical/terminal visualization: Render live matrix tables and safe sequence output in a formatted, human-readable manner after each state change.
- Implement interactive input system: Build an interactive runtime prompt to add processes, set maximums, issue requests, and display current state.
- Implement logging module: Record all safe and unsafe state transitions with timestamps to a structured log file.

### 4. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Dijkstra, E. W. (1965). Solution of a problem in concurrent programming control. *Communications of the ACM*, 8(9), 569.

3. Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson.
4. GeeksforGeeks. (2023). Banker's Algorithm in Operating System. Retrieved from <https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system-2/>